

P1873R0

remove . dots . in . module . names

Published Proposal, 2019-09-17

This version:

<http://wg21.link/D1873>

Author:

[Michael Spencer](#) (Apple)

Audience:

EWG, SG2

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

.s in module names exist in support of submodules, which we don't have. We should remove them for now, as they are likely to cause confusion and may prevent us from getting the submodules we want in the future.

Table of Contents

1 Current Semantics

2 Current Use

2.1 Implied Structure

2.2 Filesystem Mapping

3 History

4 Problems

4.1 User Confusion

4.2 Walling off the future

5 Possible Semantics

5.1 Other Languages

6 Design Tradeoffs

7 Wording

7.1 [module.unit]

References

Informative References

§ 1. Current Semantics

`.`s in module names currently have no semantics. There is no relation defined by the standard between `hello.leftpad` and `hello.rightpad` or `hello`. The only meaning they have is the implied hierarchy that developers read into them.

§ 2. Current Use

There are two current uses of `.`s in module names that I am aware of.

§ 2.1. Implied Structure

The first is to communicate with developers by implying some structure and relationship between module names with a common prefix. The general approach is that a module `m` should `export import` all modules with names that start with `m.`. For example:

```
export module std;  
export import std.vector;  
export import std.algorithm;  
...
```

§ 2.2. Filesystem Mapping

The second is for mapping module names to filesystem paths. Here `.` is used as a proxy for `/`. For example, `a.b.c` could map to `a/b/c.cppm`.

§ 3. History

Modules have had a long history in C++ dating back to at least 2004. Every paper until recently had an idea for something similar to submodules. It's useful to explore this history to see which directions we may want to go in the future.

First let's start with Daveed Vandevoorde's 2004 paper: [\[N1736\]](#)

In this paper submodules are known as module partitions. These are different from the module partitions we have today in that they are externally visible. The syntax started as `namespace << std["vector"];`, but in 2007 moved to `import std.vector;`. In this model, partitions export a subset of names from their parent module. Additionally, non-exported names from a partition are visible to any other partition of the same module. Note that this proposal also supports `::` in module names. It has no semantic meaning, and is for the purpose of allowing module names to match the namespace they define.

Next let's look at Doug Gregor's [2013 SG2 presentation](#), which also had submodules.

This was a description of clang modules and additional syntax. In this model a module exports all of its submodules as defined in a module map. Today submodules are used in clang for two primary purposes. The first is to restrict which names are visible when importing a submodule to those in the submodule. The second is to allow interdependencies between submodules without causing a cycle. Clang compiles a single module together along with all of its submodules as a single translation unit, and keeps track of which names are visible from which submodules.

Next is Microsoft's 2014 modules proposal, which also had submodules as part of the design: [\[N4214\]](#)

Section 4.1.1 Module Names and Filenames

We propose a hierarchical naming scheme for the namespace of module-name in support of submodules

Section 4.5 Submodules

A submodule can serve as cluster of translation units sharing implementation detail information (within a module) that is not meant to be accessible to outside consumers of the parent module.

The design was intended to allow control of visibility of names in submodules. This functionality never made it into the wording. The design didn't go into enough detail about how it would be implemented to determine if adding this functionality would be a breaking change.

Next we have Google's ATOM proposal: [\[P0947r1\]](#)

This proposal adds the module partitions we have today, but keeps `.` in module names. A key part of module partitions is that the partition name is not visible outside of the module in which they are defined.

§ 4. Problems

There are two main issues with keeping the `.` in module names.

§ 4.1. User Confusion

In C++ the `identifier.identifier` syntax is used by every developer. It has very specific semantic meanings, but even at the highest level it always establishes some form of hierarchy. We've already seen people be confused about C++ modules having no hierarchy, and even today a search for subpackages in Java leads to questions and answers centered on this confusion.

Developers will use `.` to communicate something to their users, but will they communicate the same thing? We will end up with different behaviors in different libraries, which will cause additional confusion.

§ 4.2. Walling off the future

By allowing `.` in module names without semantics, we potentially prevent giving them semantics in the future as it may be a breaking change.

§ 5. Possible Semantics

§ 5.1. Other Languages

Java/Groovy: `.` in package and module names only impacts where the classloader (basically the runtime dynamic linker) looks up `.class` files (Java bytecode) on the filesystem (or in `.jar`s). They have no semantic meaning in the source code.

Python: Modules cannot have `.` in their names. Instead `.`s are used for packages and subpackages (which both contain modules). Packages are determined by filesystem layout and directories are accessed using `..`. The syntax `import package.*` is controlled by the `__all__` variable in the package's `__init__.py` and can select which, if any, modules from that package are imported. Additionally, there are relative imports using `..` which go up the package hierarchy.

C#/VB.NET: No modules/packages, just namespaces. Namespaces are separated by `.` when referencing them, and are hierarchical.

JavaScript: Modules don't have names, they have paths represented by strings. Paths can contain `.`s and don't mean anything special. `/` is special as it is a directory separator.

Objective-C{,++}: Uses clang modules. Module names are separated by `.` into submodules. Submodules are used to control visibility of names and are hierarchical.

Delphi/Object Pascal: Namespaces can have `.`s in their names and there's no hierarchy.

Go: Package names cannot contain `.`, no subpackages.

Ruby: Modules in ruby are closer to namespaces, not really a modules system, and can't contain `.`s. Ruby uses library names represented by strings for loading other code, these can also be absolute paths.

Swift: Module names cannot contain `.`s. Can import Objective-C (clang) modules and submodules.

MATLAB: Package names can't contain `.`s. Subpackages are hierarchical and are accessed via `.`.

R: All identifiers can contain `.`s. `::` is used as the namespace separator.

Perl: Uses `::` and are translated to filesystem paths, can't contain `..`.

Rust: No `.` in module names. `::` is used as a crate and file system separator.

Of these only two have a `.` symbol that means something in normal code but means nothing in a package/module name. One is from the 80s and the other is from the 90s. Additionally every reference

I found for Java was either someone confused about what `.` meant, or someone explaining what it meant to people who were confused. We shouldn't follow Java's example here.

§ 6. Design Tradeoffs

When choosing a syntax there is always a design tradeoff. In this case that tradeoff has three major factors: utility, understandability, and extensibility.

There are valid usecases for `.`, as it does provide additional ways for a module author to communicate to their users a relationship between modules. We could have restricted identifiers to just `'a'` and `'b'` and have equal semantic power, but we didn't because that would severely limit communication.

We want new syntax to be understandable to existing and new C++ users. We often do this by reusing or mimicking existing syntax when it is close enough in semantics that we want that knowledge to carry over. We also choose different syntaxes when we want to avoid conflating two different concepts.

C++ cares about backwards compatibility, even in edge cases (due to Hyrum's law). When we choose a syntax we're pretty much stuck with that syntax and have great difficulty changing what it means. We should have a high bar for the benefit we get from a syntax due to this.

Given these tradeoffs, I think that for C++20 the risks to understandability and extensibility far outweigh the utility gained by allowing `.`s in module names. Over the next few years we should get to know modules better and how they are used and what changes we really want before closing this door.

§ 7. Wording

§ 7.1. [module.unit]

```
module-declaration:  
    exportopt module module-name module-partitionopt attribute-specifier-s  
module-name:  
    module-name-qualifieropt identifier  
module-partition:  
    : module-name-qualifieropt identifier  
module-name-qualifier:  
    identifier .  
module-name-qualifier identifier .
```

§ References

§ Informative References

[N1736]

Daveed Vandevoorde. [Modules in C++ \(Revision 1\)](https://wg21.link/n1736). 5 November 2004. URL: <https://wg21.link/n1736>

[N4214]

G. Dos Reis, M. Hall, G. Nishanov. [A Module System for C++ \(Revision 2\)](https://wg21.link/n4214). 13 October 2014. URL: <https://wg21.link/n4214>

[P0947r1]

Richard Smith. [Another take on Modules](https://wg21.link/p0947r1). 6 March 2018. URL: <https://wg21.link/p0947r1>